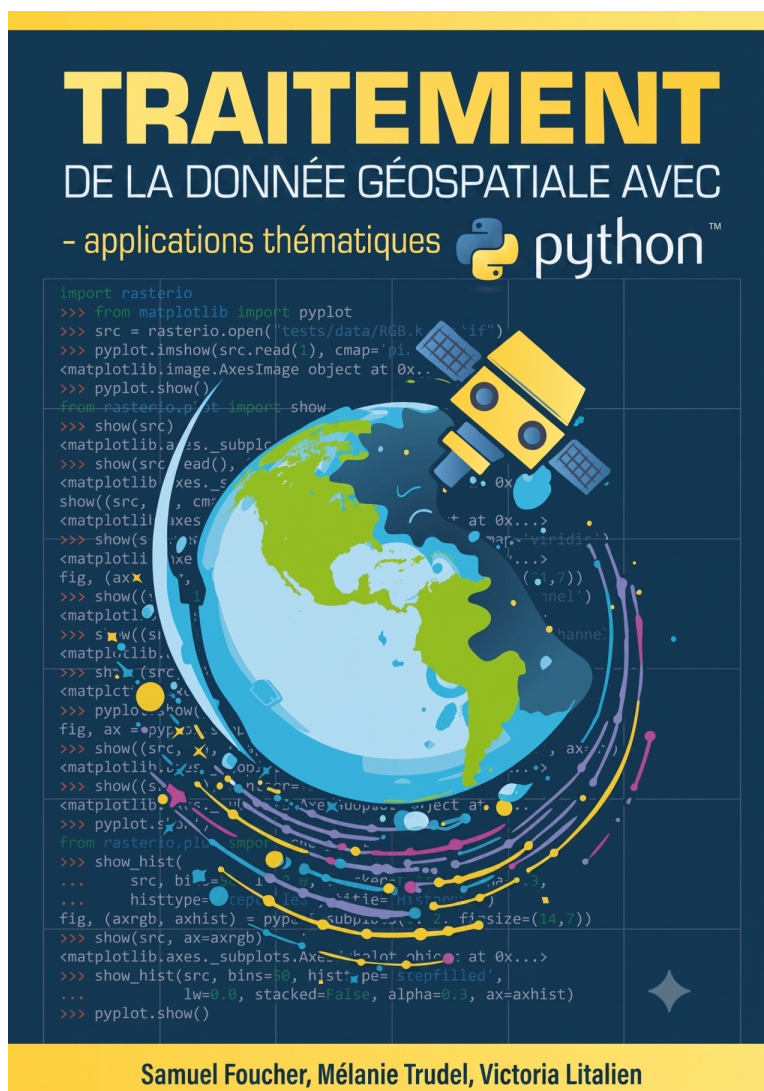




Traitement de la donnée géospatiale avec Python - applications thématiques

Version 1.14

Samuel Foucher

Mélanie Trudel

Victoria Litalien

2026-08-09

Table des matières

Préface	1
Un manuel sous la forme d’une ressource éducative libre	2
Comment lire ce manuel?	4
Comment utiliser les données du livre pour reproduire les exemples?	4
Structure du livre	5
Remerciements	5
Introduction aux images de télédétection	6
Ressources en ligne	6
Listes des <i>librairies</i> utilisés	7
À propos des auteurs	8
Partie 1. Hydrologie spatiale	9
1 Hydrologie spatiale avec la mission SWOT	10
1.1 Préambule	10
1.1.1 Objectifs	10
1.1.2 Bibliothèques	11
1.2 La mission SWOT	12
1.2.1 Objectifs de la mission	12
1.2.2 Exigences de performance	12
1.2.3 Organisation spatiale des fichiers	13
1.3 Les produits SWOT HR	13
1.3.1 L1B_HR_SLC	14
1.3.2 L2_HR_PIXC et L2_HR_PIXCVec	14
1.3.3 L2_HR_Raster	14
1.3.4 L2_HR_RiverSP	15
1.3.5 L2_HR_LakeSP	15
1.4 Systèmes de référence verticaux	16
1.5 Accès aux données	17
1.5.1 Authentification Earthdata	17
1.6 Produit rivières L2_HR_RiverSP	18
1.6.1 Recherche des granules	18
1.6.2 Téléchargement et lecture	18
1.6.3 Cartographie de l’élévation de la surface d’eau	19
1.6.4 Indicateur de qualité des nœuds	19
1.7 Produit lacs L2_HR_LakeSP	20
1.7.1 Élévation des lacs	20

1.7.2	Fraction d'eau sombre (<i>dark water</i>)	21
1.8	Produit nuage de points L2_HR_PIXC	21
1.8.1	Découpage et export vers QGIS	22
1.9	Produit L2_HR_PIXCVec	23
1.10	Produit matriciel L2_HR_Raster	24
1.10.1	Masquage par l'indicateur de qualité	24
1.10.2	Découpage sur une région d'intérêt	24
1.11	Séries temporelles avec Hydrocron	25
1.11.1	Localiser un tronçon de rivière	25
1.11.2	Série temporelle de niveau d'eau à un nœud	26
1.11.3	Série temporelle pour un lac	27
1.12	Profil en long de la surface d'eau	28
1.13	Changement de référentiel vertical avec csrspy	30
1.14	Défis et limites	31
1.15	Ressources	32
Bibliographie		34

Liste des Figures

1	Licence Creative Commons du livre	3
2	Téléchargement de l'intégralité du livre	5

Liste des Tables

1.1	Exigences de performance des produits SWOT HR	12
1.2	Les produits SWOT HR	13
1.3	Convention de nommage des attributs SWOT	15
1.4	Différences de référentiel entre SWOT et les stations au sol	16
1.5	Interprétation de l'indicateur node_q	19

Préface

Résumé : Ce troisième volume de la série aborde le traitement de la donnée géospatiale avec Python par des **applications thématiques** : chaque chapitre part d'un besoin concret (hydrologie, cryosphère, agriculture, environnement urbain, etc.) et déroule la chaîne complète, de l'accès aux données jusqu'à l'interprétation des résultats. Il suppose acquises les bases présentées dans les volumes précédents de la série. La philosophie reste la même : donner toutes les clefs de compréhension et de mise en œuvre dans Python, avec une approche compréhensive et intuitive plutôt que mathématique, sans pour autant négliger la rigueur. Plusieurs éditions régulières sont prévues sachant que ce domaine évolue constamment.

Ce projet est en cours d'écriture et le contenu n'est pas complet



Remerciements : Ce manuel a été réalisé avec le soutien de la fabriqueREL. Fondée en 2019, la fabriqueREL est portée par divers établissements d'enseignement supérieur du Québec et agit en collaboration avec les services de soutien pédagogique et les bibliothèques. Son but est de faire des ressources éducatives libres (REL) le matériel privilégié en enseignement supérieur au Québec.

Mise en page : Samuel Foucher.

© Samuel Foucher, Mélanie Trudel et Victoria Litalien.

Utilisation de l'IA : Claude Code a été utilisé pour aider sur certains aspects techniques et de production: gestion de GitHub et des profils quarto, nettoyage des notebook python, vérification du contenu et des fichiers produits, détection des erreurs.

Pour citer cet ouvrage : Foucher S., Trudel M. et Litalien V. (2026). *Traitement de la donnée géospatiale avec Python : applications thématiques*. Université de Sherbrooke, Département de géomatique appliquée. fabriqueREL. Licence CC BY-SA.



Sauf indications contraires, le contenu de ce manuel électronique est disponible en vertu des termes de la [Licence Creative Commons Attribution - Partage dans les mêmes conditions 4.0 International](#).

Vous êtes autorisé-e à :

Partager – copier, distribuer et communiquer le matériel par tous moyens et sous tous formats.

Adapter – remixer, transformer et créer à partir du matériel pour toute utilisation, y compris commerciale.

Selon les conditions suivantes :

Paternité – Vous devez citer le nom des auteurs originaux.

Mêmes conditions – Si vous remixez, transformez, ou créez à partir du matériel composant l'Œuvre originale, vous devez diffuser l'Œuvre modifiée avec la même licence.



Université de
Sherbrooke



fabrique REL
RESSOURCES ÉDUCATIVES LIBRES

Un manuel sous la forme d'une ressource éducative libre

Pourquoi un manuel sous licence libre?

Les logiciels libres sont aujourd'hui très répandus. Comparativement aux logiciels propriétaires, l'accès au code source permet à quiconque de l'utiliser, de le modifier, de le dupliquer et de le partager. Le logiciel Python, dans lequel sont mises en œuvre les méthodes de traitement d'images satellites décrites dans ce livre, est d'ailleurs à la fois un langage de programmation et un logiciel libre (sous la licence publique générale [GNU GPL2](#)). Par analogie aux logiciels libres, il existe aussi des **ressources éducatives libres (REL)** « dont la licence accorde les permissions désignées par les 5R (**R**etenir — **R**éutiliser — **R**éviser — **R**emixer — **R**edistribuer) et donc permet nécessairement la modification » ([fabriqueREL](#)). La licence de ce livre, CC BY-SA (figure 1), permet donc de :

- **Retenir**, c'est-à-dire télécharger et imprimer gratuitement le livre. Notez qu'il aurait été plutôt surprenant d'écrire un livre payant sur un logiciel libre et donc gratuit. Aussi, nous aurions été très embarrassés que des personnes étudiantes avec des ressources financières limitées doivent payer pour avoir accès au livre, sans pour autant savoir préalablement si le contenu est réellement adapté à leurs besoins.
- **Réutiliser**, c'est-à-dire utiliser la totalité ou une section du livre sans limitation et sans compensation financière. Cela permet ainsi à d'autres personnes enseignantes de l'utiliser dans le cadre d'activités pédagogiques.
- **Réviser**, c'est-à-dire modifier, adapter et traduire le contenu en fonction d'un besoin pédagogique précis puisqu'aucun manuel n'est parfait, tant s'en faut! Le livre a d'ailleurs été écrit intégralement dans Python avec [Quatro](#). Quiconque peut ainsi télécharger gratuitement le code source du livre sur [GitHub](#) et le modifier à sa guise (voir l'encadré intitulé *Suggestions d'adaptation du manuel*).
- **Remixer**, c'est-à-dire « combiner la ressource avec d'autres ressources dont la licence le permet aussi pour créer une nouvelle ressource intégrée » ([fabriqueREL](#)).
- **Redistribuer**, c'est-à-dire distribuer, en totalité ou en partie le manuel ou une version révisée sur d'autres canaux que le site Web du livre (par exemple, sur le site Moodle de votre université ou en faire une version imprimée).

La licence de ce livre, CC BY-SA (figure 1), oblige donc à :

- Attribuer la paternité de l'auteur dans vos versions dérivées, ainsi qu'une mention concernant les grandes modifications apportées, en utilisant la formulation suivante :

Samuel Foucher, Mélanie Trudel et Victoria Litalien (2026). *Traitement de la donnée géospatiale avec Python : applications thématiques*. Université de Sherbrooke, Département de géomatique appliquée. fabriqueREL. Licence CC BY-SA.

- Utiliser la même licence ou une licence similaire à toutes versions dérivées.

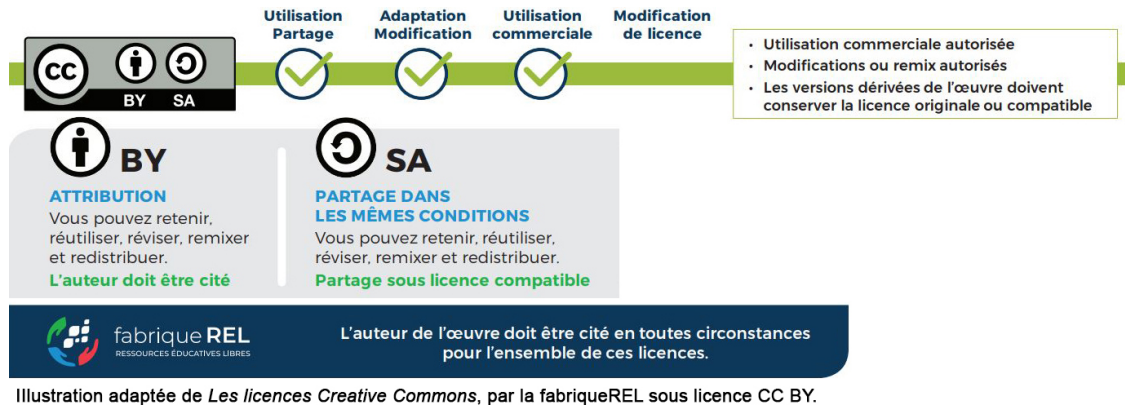


FIGURE 1 – Licence Creative Commons du livre

💡 Astuce

Suggestions d'adaptation du manuel

Pour chaque méthode de traitement d'image abordée dans le livre, une description détaillée et une mise en œuvre dans Python sont disponibles. Par conséquent, plusieurs adaptations du manuel sont possibles :

- Conserver uniquement les chapitres sur les méthodes ciblées dans votre cours.
- En faire une version imprimée et la distribuer aux personnes étudiantes.
- Modifier la description d'une ou de plusieurs méthodes en effectuant les mises à jour directement dans les chapitres.
- Insérer ses propres jeux de données dans les sections intitulées *Mise en œuvre dans Python*.
- Modifier les tableaux et figures.
- Ajouter une série d'exercices.
- Modifier les quiz de révision.
- Rédiger un nouveau chapitre.
- Modifier des syntaxes en Python. Plusieurs *librairies* Python peuvent être utilisées pour mettre en œuvre telle ou telle méthode. Ces derniers évoluent aussi très vite et de nouvelles *librairies* sont proposées fréquemment! Par conséquent, il peut être judicieux de modifier une syntaxe Python du livre en fonction de ses habitudes de programmation en Python (utilisation d'autres *librairies* que ceux utilisés dans le manuel par exemple) ou de bien mettre à jour une syntaxe à la suite de la parution d'une nouvelle *librairie* plus performante ou intéressante.
- Toute autre adaptation qui permet de répondre au mieux à un besoin pédagogique.

Comment lire ce manuel?

Le livre comprend plusieurs types de blocs de texte qui en facilitent la lecture.

Package

Bloc *packages*

Habituellement localisé au début d'un chapitre, il comprend la liste des *packages* Python utilisés pour un chapitre.

Objectif

Bloc objectif

Il comprend une description des objectifs d'un chapitre ou d'une section.

Note

Bloc notes

Il comprend une information secondaire sur une notion, une idée abordée dans une section.

Aller plus loin

Bloc pour aller plus loin

Il comprend des références ou des extensions d'une méthode abordée dans une section.

Astuce

Bloc astuce

Il décrit un élément qui vous facilitera la vie : une propriété statistique, un *package*, une fonction, une syntaxe Python.

Attention

Bloc attention

Il comprend une notion ou un élément important à bien maîtriser.

Exercice

Bloc exercice

Il comprend un court exercice de révision à la fin de chaque chapitre.

Comment utiliser les données du livre pour reproduire les exemples?

Ce livre comprend des exemples détaillés et appliqués en Python pour chacune des méthodes abordées. Ces exemples se basent sur des jeux de données ouverts et mis à disposition avec le livre. Ils sont disponibles sur le *repo GitHub* dans le sous-dossier **data**, à l'adresse <https://github.com/serie-tele-pyton/TraitementImagesPythonVol3/tree/main/data>.

Une autre option est de télécharger le *repo* complet du livre directement sur *GitHub* (<https://github.com/serie-tele-pyton/TraitementImagesPythonVol3>) en cliquant sur le bouton **Code**, puis le bouton **Download ZIP** (figure 2). Les données se trouvent alors dans le sous-dossier nommé **data**.

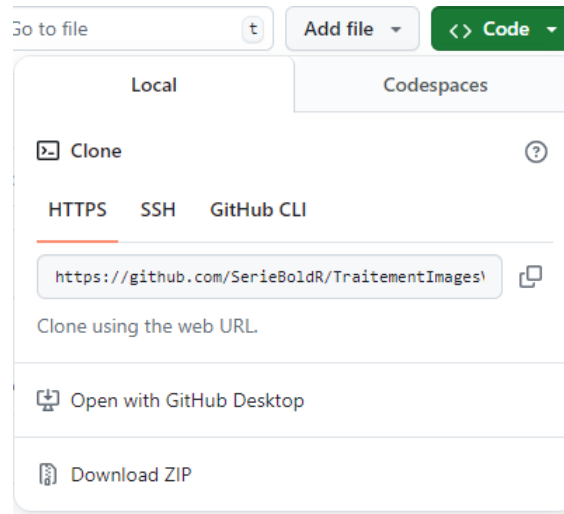


FIGURE 2 – Téléchargement de l'intégralité du livre

Structure du livre

Le livre est organisé par grandes thématiques d'application, chacune regroupant un ou plusieurs chapitres. D'autres parties viendront s'ajouter au fil des éditions.

Partie 1. Hydrologie spatiale. Cette première partie est consacrée au suivi des eaux de surface depuis l'espace. Le chapitre sur la mission **SWOT** (chapitre 1) couvre l'ensemble de la chaîne : la mission et son instrument **KaRIn**, les six produits hydrologiques haute résolution, l'accès aux données avec **earthaccess**, la lecture des produits vectoriels et matriciels, le filtrage par les indicateurs de qualité, l'extraction de séries temporelles avec **Hydrocron**, le tracé de profils en long et la conversion vers les référentiels verticaux canadiens.

Remerciements

De nombreuses personnes ont contribué à l'élaboration de ce manuel.

Ce projet a bénéficié du soutien pédagogique et financier de la **fabriqueREL** (ressources éducatives libres). Les différentes rencontres avec le comité de suivi nous ont permis de comprendre l'univers des ressources éducatives libres (REL) et notamment leurs **fameux 5R** (Retenir — Réutiliser — Réviser — Remixer — Redistribuer), de mieux définir le besoin pédagogique visé par ce manuel, d'identifier des ressources pédagogiques et des outils pertinents pour son élaboration. Ainsi, nous remercions chaleureusement les membres de la **fabriqueREL** pour leur soutien inconditionnel :

- José-Miguel Escobar-Zuniga, bibliothécaire à l'Université de Sherbrooke.
- Marianne Dubé, coordonnatrice de la **fabriqueREL**, Université de Sherbrooke.

— Claude Potvin, conseiller en formation, Service de soutien à l’enseignement, Université Laval.

Nous remercions chaleureusement les personnes étudiantes du [Baccalauréat en géomatique appliquée à l’environnement](#) et du [Microprogramme de 1er cycle en géomatique appliquée](#) du [Département de géomatique appliquée](#) de l’[Université de Sherbrooke](#) qui utilisent le manuel dans le cadre de cours.

Introduction aux images de télédétection

L’imagerie numérique a pris une place importante dans notre vie de tous les jours depuis une quinzaine d’années. Ces images sont prises généralement au niveau du sol (imagerie proximale) avec seulement trois couleurs dans le domaine de la vision humaine (rouge, vert et bleu). Dans la suite du manuel, on parlera d’images du domaine de la vision par ordinateur ou images en vision pour faire plus court.

Les images de télédétection ont des particularités et des propriétés qui les différencient des images “classiques”, dont les cinq principales sont:

1. Les images sont géoréférencées. Cela signifie que pour chaque pixel nous pouvons y associer une position géographique ou cartographique.
2. Le point de vue est très différent. Ces images sont prises avec une vue d’en haut (Nadir) ou oblique avec une distance qui peut être très grande; on parle d’images distales.
3. Elles possèdent plus que 3 bandes. Contrairement aux images en vision, les images de télédétection possèdent bien souvent plus que trois bandes. Il n’est pas rare de trouver quatre bandes (Pléiade), 13 bandes (Sentinel-2, Landsat) et même 200 bandes pour des capteurs hyperspectraux.
4. Elles peuvent être calibrées. Les valeurs numérique de l’image peuvent être converties en quantités physiques (luminance, réflectance, section efficace, etc.) via une fonction de calibration.
5. Elles sont de grande taille. Il n’est pas rare de manipuler des images qui font plusieurs dizaines de milliers de pixels en dimension.

Ressources en ligne

Voici une sélection de ressources gratuites, en accès libre, pour approfondir les notions abordées dans ce manuel.

Python et NumPy

- [Le tutoriel officiel de Python](#) (en français);
- [NumPy — the absolute basics for beginners](#);
- [NumPy Tutorials](#) — tutoriels appliqués (traitement d’images, algèbre linéaire, etc.);
- [Python NumPy Tutorial \(CS231n, Stanford\)](#).

Données géospatiales et matricielles

- [Introduction to GIS Programming \(geog-312\)](#) — cours complet en Python ([rasterio](#), [rioxarray](#), [xarray](#), [leafmap](#)...);
- [Xarray Tutorial](#) — le tutoriel officiel de [xarray](#);
- [Documentation de rioxarray](#);
- [Documentation de Rasterio](#);
- [GDAL](#) — la librairie de référence pour les formats géospatiaux.

Visualisation

- [Tutoriels Matplotlib](#);
- [leafmap](#) — cartes interactives pour l'analyse géospatiale.

Indices spectraux et apprentissage automatique

- [Awesome Spectral Indices](#) (librairie `spyndex`);
- [scikit-learn](#) — apprentissage automatique en Python;
- [scikit-image](#) — traitement d'images.

Listes des *librairies* utilisés

Dans ce livre, nous utilisons de nombreux *packages* Python. La liste des bibliothèques nécessaires est donnée au début de chaque chapitre, dans le bloc *packages*.

À propos des auteurs

Samuel Foucher est professeur au Département de géomatique appliquée de l'Université de Sherbrooke. Il y enseigne aux programmes de 1^{er} et 2^e cycles de géomatique les cours *Traitement numérique des images de télédétection*, *Base de données géospatiales* et *Apprentissage profond appliqué à l'observation de la Terre*. Ses intérêts de recherche portent sur le traitement d'images et l'application de l'IA aux données géospatiales.

Mélanie Trudel est professeure au Département de génie civil et de génie du bâtiment de la Faculté de génie de l'Université de Sherbrooke. Elle détient un doctorat en génie civil de l'École de technologie supérieure et une maîtrise en télédétection de l'Université de Sherbrooke. Ses intérêts de recherche portent sur la télédétection appliquée aux ressources en eau, ainsi que sur la modélisation hydrologique et hydraulique.

Gabriela Siles est professeure adjointe au Département des sciences géomatiques de l'Université Laval. Elle détient un doctorat en ingénierie de la *Technische Universität Braunschweig* (Allemagne), un diplôme d'ingénieure en géodésie et géophysique de l'*Universidad Nacional de Tucumán* (Argentine), et a réalisé un stage postdoctoral en télédétection et géomatique à l'Université de Sherbrooke. Ses expertises couvrent la télédétection et le traitement d'images, l'acquisition de données géospatiales et hydrospatiales, ainsi que l'analyse et la modélisation spatiales appliquées aux ressources en eau. Elle est membre de l'équipe scientifique de la mission SWOT.

Victoria Litalien est professionnelle de recherche au Département de génie civil et de génie du bâtiment de l'Université de Sherbrooke. Elle détient une maîtrise en géomatique appliquée et télédétection de l'Université de Sherbrooke. Ses travaux portent sur la télédétection, les ressources en eau et le suivi du couvert de glace des rivières.

Partie 1. Hydrologie spatiale

1 Hydrologie spatiale avec la mission SWOT

Mélanie Trudel, Gabriela Siles, Victoria Litalien, Samuel Foucher

1.1 Préambule

Assurez-vous de lire ce préambule avant d'exécuter le reste du notebook.

1.1.1 Objectifs

Ce chapitre présente la mission satellitaire **SWOT** (*Surface Water and Ocean Topography*) et montre comment accéder, filtrer et visualiser ses produits hydrologiques haute résolution (HR) avec Python. Ce chapitre est aussi disponible sous la forme d'un notebook Python:



🎯 Objectif

Objectifs d'apprentissage visés dans ce chapitre

À la fin de ce chapitre, vous devriez être en mesure de :

- décrire la mission SWOT, son instrument **KaRIn** et ses exigences de performance;
- distinguer les six produits SWOT HR (**SLC**, **PIXC**, **PIXCVec**, **Raster**, **RiverSP**, **LakeSP**) et savoir lequel choisir selon le besoin;
- comprendre l'organisation spatiale des fichiers (passe, tuile, scène, continent) et la nomenclature des granules;
- vous authentifier auprès de **NASA Earthdata** et rechercher des granules avec **earthaccess**;
- lire des produits vectoriels (**shapefile**) avec **geopandas** et des produits matriciels (**NetCDF**) avec **xarray** et **rioxarray**;
- filtrer les données à l'aide des **indicateurs de qualité** (**node_q**, **wse_qual**, **dark_frac**, **ice_clim_f**) et des valeurs de remplissage;
- découper une emprise d'intérêt, exporter vers **shapefile**/**NetCDF** pour QGIS;
- extraire des **séries temporelles** de niveau d'eau avec l'API **Hydrocron**;
- tracer un **profil en long** de la surface d'eau entre deux nœuds d'un cours d'eau;
- comprendre la différence entre hauteur **ellipsoïdale** et hauteur **orthométrique** et convertir les élévations SWOT vers un référentiel vertical canadien;
- identifier les principales sources d'erreur (*dark water*, *layover*, géolocalisation, bases a priori, glace).

1.1.2 Bibliothèques

Les bibliothèques utilisées dans ce chapitre sont les suivantes:

- [earthaccess](#) — recherche et téléchargement des données NASA Earthdata;
- [GeoPandas](#) — lecture et manipulation des **shapefile**;
- [Xarray](#) et [rioxarray](#) — lecture des **NetCDF** et découpage géospatial;
- [Shapely](#) — géométries et emprises;
- [contextily](#) — fonds de carte;
- [hvPlot](#) / [HoloViews](#) — visualisation interactive;
- [Folium](#) — cartes interactives;
- [Matplotlib](#) — figures statiques;
- [csrspy](#) — transformations de référentiels géodésiques canadiens.

Package

Installation

Dans l'environnement Google Colab, les bibliothèques listées ci-dessus doivent être installées avant la première exécution (cellule suivante).

```
1 !pip install -q earthaccess geopandas rioxarray rasterio contextily shapely pyproj python-dotenv
2 !pip install -q h5netcdf holoviews hvplot bokeh geoviews folium tqdm
```

Vérifier les importations:

```
1 import os
2 import glob
3 import zipfile
4 import logging
5 from datetime import datetime
6 from io import StringIO
7
8 import numpy as np
9 import pandas as pd
10 import geopandas as gpd
11 import xarray as xr
12 import rioxarray
13 import matplotlib.pyplot as plt
14 import matplotlib.colors as mcolors
15 import contextily as cx
16 import requests
17 import folium
18 import earthaccess
19 import holoviews as hv
20 import hvplot.xarray
21 from shapely.geometry import box, mapping, Point
22 from tqdm import tqdm
```


1.2 La mission SWOT

SWOT (*Surface Water and Ocean Topography*) est une mission d'altimétrie satellitaire issue d'un partenariat entre la **NASA** et le **CNES**, avec des contributions de l'**Agence spatiale canadienne** (ASC) et de l'agence britannique (UKSA). Lancée en **décembre 2022**, elle est prévue pour une durée de vie de trois à cinq ans.

Son instrument principal est le **KaRIn** (*Ka-band Radar Interferometer*), un interféromètre radar en bande Ka qui observe la surface à des angles d'incidence proches du nadir. Contrairement à un altimètre nadir classique qui ne mesure qu'une trace au sol, **KaRIn** produit une **fauchée** de part et d'autre de la trace, ce qui permet une cartographie bidimensionnelle des hauteurs d'eau.

Le satellite repasse au-dessus d'un même point tous les **21 jours** (orbite dite *Science*), avec la possibilité de plusieurs passes sur un même plan d'eau à l'intérieur d'un cycle selon la latitude.

1.2.1 Objectifs de la mission

- dresser un inventaire global de **tous les plans d'eau continentaux** d'une superficie supérieure à $(250\text{ m})^2$ (objectif : $(100\text{ m})^2$, seuil minimal : 1 km^2) — lacs, réservoirs et milieux humides;
- dresser un inventaire global des **rivières de plus de 100 m de largeur** (objectif : 50 m, seuil : 170 m);
- mesurer les changements de **stockage d'eau** dans les plans d'eau continentaux aux échelles infra-mensuelle, saisonnière et annuelle;
- estimer les changements de **débit** des rivières aux mêmes échelles temporelles.

1.2.2 Exigences de performance

Les exigences sont exprimées à 1σ , c'est-à-dire l'intervalle qui contient 68 % des valeurs pour une distribution gaussienne. Attention à ne pas confondre **justesse** (absence de biais) et **précision** (faible dispersion) : un capteur peut être précis sans être juste.

TABLEAU 1.1 – Exigences de performance des produits SWOT HR

Grandeur	Exigence (1σ)
Élévation de la surface d'eau (WSE), plans d'eau de plus de 1 km^2	10 cm ou mieux
WSE, plans d'eau entre $(250\text{ m})^2$ et 1 km^2	25 cm ou mieux
Superficie (erreur relative), plans d'eau de plus de $(250\text{ m})^2$	moins de 15 %
Pente des rivières de plus de 100 m de largeur	$17\text{ }\mu\text{rad}$ (1,7 cm/km), moyennée sur au plus 10 km

Note**Versions et retraitements**

Les produits SWOT évoluent au fil des retraitements. La chaîne de production distingue les traitements **avant** (*forward*, notés **PIC0**, **PIC2**, **PID0**, disponibles quelques jours après l’acquisition) des **retraitements globaux** (**PGC0**, **PGD0**). Au moment d’écrire ces lignes, la version D (**PID0**) est diffusée depuis le 6 mai 2025 et un retraitement global **PGD0** est en cours.

Le nom abrégé (*short name*) de la collection change avec la version : **SWOT_L2_HR_RiverSP_2.0** pour la version C, **SWOT_L2_HR_RiverSP_D** pour la version D. **Les identifiants de nœuds et de tronçons ne sont pas identiques d’une version à l’autre.** Consultez toujours les [notes de version](#) avant d’exploiter un jeu de données.

1.2.3 Organisation spatiale des fichiers

Le découpage des produits SWOT HR repose sur quatre notions :

- la **pas**se (*pass*, numérotée de 1 à 584) : SWOT suit exactement la même trace au sol tous les 21 jours, sur 292 orbites. Chaque orbite comporte une passe ascendante (vers le nord) et une passe descendante (vers le sud);
- la **fauchée** (*swath*) : la surface couverte par **KaRIn**. Aucune donnée exploitable n’est disponible à moins de 20 km du nadir ni au-delà de 64 km;
- la **tuile** (*tile*) : chaque passe est découpée en tuiles de 64×64 km, une à gauche (**L**) et une à droite (**R**) du nadir;
- la **scène** (*scene*) : un regroupement de 4 tuiles (2 **L** et 2 **R**), soit 128×128 km.

Les produits vectoriels rivières et lacs sont quant à eux découpés par **continent** (**NA** pour l’Amérique du Nord). Le dépôt [SWOT-Canada](#) fournit les couches permettant de retrouver la passe, la tuile ou la scène correspondant à une zone d’intérêt.

1.3 Les produits SWOT HR

Les produits SWOT HR se répartissent sur un continuum allant des produits **experts** (proches de la mesure radar brute) aux produits **conviviaux** (agrégés sur des objets hydrographiques).

TABLEAU 1.2 – Les produits SWOT HR

Produit	Type	Format	Découpage	Public
L1B_HR_SLC	Image complexe en géométrie radar	NetCDF4	Tuile	Expert
L2_HR_PIXC	Nuage de points géolocalisés	NetCDF4	Tuile	Expert
L2_HR_PIXCVec	Nuage de points amélioré	NetCDF4	Tuile	Expert

Produit	Type	Format	Découpage	Public
L2_HR_Raster	Grille régulière (WSE, étendue, rétrodiffusion)	NetCDF4	Scène	Convivial
L2_HR_RiverSP	Tronçons (lignes) et nœuds (points) de rivières	Shapefile	Continent	Convivial
L2_HR_LakeSP	Polygones de lacs	Shapefile	Continent	Convivial

Deux **bases de données a priori** sous-tendent les produits vectoriels :

- **SWORD** (*SWOT River Database*, ou PRD) : le réseau hydrographique mondial découpé en tronçons (lignes) et en nœuds (points) espacés d'environ 200 m. Voir **SWORD** et Altenau *et al.* (2021), <https://doi.org/10.1029/2021WR030054>;
- **PLD** (*Prior Lake Database*) : le masque mondial des lacs sous forme de polygones. Voir **Hydroweb.next** et Wang *et al.* (2025), <https://doi.org/10.1029/2023WR036896>.

1.3.1 L1B_HR_SLC

Le produit **SLC** (*Single Look Complex*) est l'image dans le plan oblique du radar (*slant range*). Elle représente les données telles que mesurées, selon l'angle de visée du satellite : elle suit la trajectoire du satellite et **n'est donc pas géolocalisée**. Ce produit s'adresse aux personnes qui souhaitent appliquer leurs propres algorithmes de traitement interférométrique. Un fichier NetCDF comporte cinq groupes (**slc**, **xfactor**, **noise**, **tv**, **grdem**) et contient notamment la longueur d'onde, la polarisation, les canaux **slc_plus_y** / **slc_minus_y**, l'attitude du satellite (**roll**, **pitch**, **yaw**) et des indicateurs de qualité.

1.3.2 L2_HR_PIXC et L2_HR_PIXVec

Le produit **PIXC** (*Water Mask Pixel Cloud*) contient les **mesures de hauteur géolocalisées** dérivées du **SLC**, sous forme d'un nuage de points, avec une **classification** pour chaque pixel. Il est organisé en trois groupes : **pixel_cloud**, **tv** et **noise**. Le groupe **pixel_cloud** fournit entre autres la classification, la hauteur (**ellipsoïdale**, WGS84), **water_area**, **water_frac**, la rétrodiffusion **sig0**, la distance signée au nadir **cross_track** et l'angle d'incidence.

Le produit **PIXVec** est dérivé du **PIXC**. Il sert d'étape intermédiaire dans la construction des produits rivières et lacs et fournit une **meilleure localisation des pixels** ainsi que l'information sur la glace.

1.3.3 L2_HR_Raster

Le produit **Raster** fournit la WSE (référéncée sur le géoïde **EGM2008**), l'étendue inondée et la rétrodiffusion sur une **grille régulière** en 2D, découpée en scènes. Deux résolutions sont disponibles : 100 m et 250 m. Un fichier NetCDF contient à la fois une grille UTM et une grille géodésique latitude/longitude. C'est le produit le plus simple à ouvrir directement dans QGIS.

1.3.4 L2_HR_RiverSP

Un algorithme rattache les pixels pertinents à chaque **tronçon** (*reach*) et **nœud** (*node*) de la base SWORD, puis agrège les mesures. Le produit est diffusé sous forme de deux **shapefile** (**Reach** et **Node**) par continent, contenant les identifiants, la WSE (orthométrique, **EGM2008**), la superficie, la largeur, la pente, la distance au nadir **xtrk_dist**, l'ondulation du géoïde **geoid_hght** et de nombreux indicateurs de qualité.

Les suffixes et préfixes des attributs suivent une convention systématique :

TABLEAU 1.3 – Convention de nommage des attributs SWOT

Marqueur	Signification
p_	information issue de la base a priori (PLD/SWORD)
_c	correction
_f	indicateur (<i>flag</i>)
_q	indicateur de qualité
_u	incertitude (1σ , ou 68 ^e percentile)
_std	écart-type à l'intérieur du lac

Comme SWOT mesure simultanément la largeur, l'élévation et la pente d'une rivière, il devient possible d'en **estimer le débit** à l'aide de relations hydrauliques. Plusieurs algorithmes coexistent (MetroMan, BAM, HiVDI, MOMMA, SADS, SIC4DVar); voir Durand *et al.* (2023), <https://doi.org/10.1029/2021WR031614>.

⚠ Attention

Débits et versions

Les estimations de débit ne sont **pas fournies** dans les produits **PID0** et **PGD0**; elles le seront après le retraitement complet de la version D. Un produit de niveau 4 dédié au débit est toutefois disponible depuis le 1^{er} décembre 2025 : **SWOT_L4_HR_DAWG_SOS_DISCHARGE_V3**.

1.3.5 L2_HR_LakeSP

Pour les lacs, l'algorithme sélectionne les pixels **PIXC** correspondant à chaque lac de la base PLD et moyenne les mesures. Trois types de fichiers sont produits :

- **Obs** : les entités effectivement observées (polygones tels que vus par SWOT);
- **Prior** : les entités de la base PLD, renseignées avec les observations (une entité non observée reste vide);
- **Unassigned** : les entités observées qui n'ont pu être rattachées à aucun lac de la base.

Autrement dit : **Obs** est **ce que l'on voit**, **Prior** est **ce que l'on attendait**, et **Unassigned** est **ce qui ne correspond pas à la base**. Pour un suivi temporel d'un lac donné, c'est le fichier **Prior** qu'il faut privilégier, puisque l'identifiant **lake_id** y est stable dans le temps.

1.4 Systèmes de référence verticaux

Lorsqu'on parle d'élévation, deux surfaces de référence coexistent :

- l'**ellipsoïde** est une approximation mathématique de la forme de la Terre. Les élévations mesurées par rapport à l'ellipsoïde sont des **hauteurs ellipsoïdales** (h);
- le **géoïde** est un modèle du champ de pesanteur terrestre, approximativement le niveau moyen des mers. Les élévations mesurées par rapport au géoïde sont des **hauteurs orthométriques** (H).

Astuce

height ou wse?

Dans les produits SWOT, l'attribut **height** désigne une hauteur **ellipsoïdale** et l'attribut **wse** une hauteur **orthométrique** (géoïde). Quand vous lisez « height », pensez ellipsoïde; quand vous lisez « WSE », pensez géoïde.

La relation générale est $H = h - N$, où N est l'ondulation du géoïde. Le champ **geoid_hght** des produits SWOT contient l'ondulation du géoïde **EGM2008** en convention *mean-tide* (marée permanente incluse). On revient donc à la hauteur ellipsoïdale par :

$$h_{\text{SWOT}} = \text{WSE}_{\text{SWOT}} + \text{geoid_hght}$$

Comparer une mesure SWOT à une station hydrométrique canadienne exige de réconcilier trois différences : le **datum**, l'**époque** et le **géoïde**.

TABLEAU 1.4 – Différences de référentiel entre SWOT et les stations au sol

	Station (Québec)	Station (Canada : NB, NS, NL)	WSE SWOT
Datum	NAD83 (SCRS)	NAD83 (SCRS)	ITRF2014
Époque	1997	2010	date d'acquisition
Géoïde	CGVD1928 (HT2)	CGVD2013	EGM2008
Marée	<i>tide-free</i>	<i>tide-free</i>	<i>mean-tide</i>

La chaîne de conversion est donc : (1) passer de la WSE SWOT à la hauteur ellipsoïdale, (2) transformer h_{ITRF2014} , époque d'acquisition vers $h_{\text{NAD83(SCRS)}}$, époque de la station, (3) retrancher le géoïde de la station. La section 1.13 automatise ces étapes avec **csrspy**.

Attention

Chaque province a son référentiel

Chaque agence géodésique provinciale canadienne peut adopter un système de référence vertical et une époque différents. Découpez toujours vos données par zone d'intérêt **avant** la transformation, et vérifiez le référentiel en vigueur dans la province concernée.

1.5 Accès aux données

Plusieurs portails permettent d'explorer les données SWOT sans écrire de code :

- [Earthdata Search](#) — le portail de référence de la NASA (recherche par emprise, par période et par filtre sur le nom du granule);
- [Hydroweb.next](#) — le portail du pôle Theia (CNES);
- [SWOTvis](#) — visualisation de séries temporelles (CUAHSI);
- [WISP](#) — l'explorateur de l'USGS;
- [Swath visualizer](#) — pour repérer la passe et la date de survol d'une zone.

Dans la suite, nous privilégions l'accès programmatique avec **earthaccess**, qui permet de combiner plusieurs produits dans un même script.

1.5.1 Authentification Earthdata

Un compte **Earthdata Login** est nécessaire pour accéder aux données de la NASA. La création du compte est gratuite : <https://urs.earthdata.nasa.gov>. La bibliothèque **earthaccess** gère l'authentification et le renouvellement des jetons.

La meilleure pratique consiste à stocker le jeton dans un fichier `.env` à la racine du projet, puis à le charger avec **python-dotenv**. Le code suivant crée le fichier `.env` et y écrit votre jeton. Remplacez `'votre_token_ici'` par votre jeton personnel.

```
1 %%writefile .env
2 EARTHDATA_TOKEN='votre_token_ici'
```

Le fichier est ensuite chargé et le jeton est utilisé pour se connecter à Earthdata. Si le jeton n'est pas défini ou est laissé à la valeur par défaut, le code retombe sur la méthode de connexion originale, qui peut demander vos identifiants.

```
1 from dotenv import dotenv_values, load_dotenv
2 import os
3
4 # Load the .env file
5 load_dotenv()
6
7 # Retrieve the token from environment variables
8 token = os.getenv('EARTHDATA_TOKEN')
9
10 if token and token != 'votre_token_ici':
11     print("Token loaded from .env file successfully!")
12     # Now you can use this token to log in
13     auth = earthaccess.login(strategy='token')
14     print("Successfully logged in to Earthdata using token from .env file!")
15 else:
```

```

16 print("Please replace 'votre_token_ici' in the .env file with your actual Earthdata token.")
17 print("Falling back to original login attempt (which might prompt for credentials or use existing
   ↪ env var if set).")
18 # Fallback to the original login method if the .env token is not set or is placeholder
19 auth = earthaccess.login()

```

1.6 Produit rivières L2_HR_RiverSP

1.6.1 Recherche des granules

La fonction `earthaccess.search_data` interroge le catalogue à partir du nom abrégé de la collection. Comme cette collection contient à la fois les fichiers **Reach** et **Node**, on filtre sur le nom du granule. Le motif `*Node*_214_NA*` sélectionne les fichiers de nœuds, de la passe 214, pour l'Amérique du Nord.

```

1 river_results = earthaccess.search_data(
2     short_name='SWOT_L2_HR_RiverSP_D', # 'SWOT_L2_HR_RiverSP_2.0' pour la version C
3     # temporal=('2025-01-01 00:00:00', '2026-01-31 23:59:59'),
4     granule_name='*Node*_214_NA*') # 'Node' ou 'Reach', numéro de passe, continent
5
6 print(river_results)

```

1.6.2 Téléchargement et lecture

`earthaccess.download` attend une **liste** de granules; on encadre donc de crochets le granule choisi. Ici, on prend le plus récent de la collection.

```

1 earthaccess.download([river_results[-1]], "./data_downloads")

```

Le format natif est une archive `.zip` qu'il faut décompresser pour accéder au `.shp`. On récupère d'abord le nom de fichier depuis le lien du granule.

```

1 filename = earthaccess.results.DataGranule.data_links(river_results[-1], access='external')
2 filename = filename[0].split("/)[-1]
3
4 with zipfile.ZipFile(f'data_downloads/{filename}', 'r') as zip_ref:
5     zip_ref.extractall('data_downloads')

```

La table attributaire se lit ensuite avec `geopandas`.

```

1 filename_shp = filename.replace('.zip', '.shp')
2 SWOT_HR_shp1 = gpd.read_file(f'data_downloads/{filename_shp}')
3 SWOT_HR_shp1

```

1.6.3 Cartographie de l'élévation de la surface d'eau

Deux précautions avant de cartographier : **filtrer la valeur de remplissage** -999999999999 (utilisée pour les nœuds non mesurés) et **découper** sur une emprise d'intérêt, sans quoi la figure couvre tout le continent.

```

1 # Emprise d'intérêt (xmin, ymin, xmax, ymax) : région de Fredericton (N.-B.)
2 bounding_box = box(-67.01, 45.82, -66.42, 46.4)
3
4 # Écarter les valeurs de remplissage de la colonne 'wse'
5 SWOT_HR_shp1_filtered = SWOT_HR_shp1[SWOT_HR_shp1['wse'] != -999999999999]
6 SWOT_HR_shp1_clipped = gpd.clip(SWOT_HR_shp1_filtered, bounding_box)
7
8 fig, ax = plt.subplots(figsize=(10, 10))
9 SWOT_HR_shp1_clipped.plot(
10     ax=ax, column='wse', cmap='cool', legend=True,
11     legend_kwds={'label': "Élévation de la surface d'eau (m)", 'orientation': "vertical"})
12 cx.add_basemap(ax, crs=SWOT_HR_shp1_clipped.crs, source=cx.providers.Esri.WorldStreetMap)
13 plt.show()

```

1.6.4 Indicateur de qualité des nœuds

L'attribut `node_q` résume la qualité d'un nœud sur quatre niveaux :

TABLEAU 1.5 – Interprétation de l'indicateur `node_q`

Valeur	Signification
0	bonne (<i>good</i>)
1	suspecte (<i>suspect</i>) — peut comporter des erreurs importantes
2	dégradée (<i>degraded</i>) — comporte très probablement des erreurs importantes
3	mauvaise (<i>bad</i>) — potentiellement aberrante, à ignorer

Comme il s'agit d'une variable **discrète**, on construit une palette et une normalisation par classes plutôt qu'un dégradé continu.

```

1 unique_values = np.sort(SWOT_HR_shp1['node_q'].unique())
2
3 # Palette discrète : une couleur par classe de qualité
4 cmap = plt.get_cmap('RdYlGn_r', len(unique_values))
5 norm = mcolors.BoundaryNorm(
6     boundaries=np.arange(len(unique_values) + 1) - 0.5, ncolors=len(unique_values))
7
8 fig, ax = plt.subplots(figsize=(10, 10))

```



```

9 SWOT_HR_shp1.plot(ax=ax, column='node_q', cmap=cmap, norm=norm, legend=False)
10
11 sm = plt.cm.ScalarMappable(cmap=cmap, norm=norm)
12 sm.set_array([])
13 cbar = plt.colorbar(sm, ax=ax, ticks=range(len(unique_values)))
14 cbar.ax.set_yticklabels(unique_values)
15 cbar.set_label('Qualité des nœuds (node_q)')
16
17 # Restreindre l'affichage à la zone d'intérêt
18 ax.set_xlim(-67.01, -66.42)
19 ax.set_ylim(45.82, 46.4)
20 cx.add_basemap(ax, crs=SWOT_HR_shp1.crs, source=cx.providers.Esri.WorldStreetMap)
21 plt.show()

```

1.7 Produit lacs L2_HR_LakeSP

Le produit lacs s'obtient exactement de la même manière. Le motif `*Prior*_214_NA*` sélectionne ici le fichier `Prior`, celui qu'il faut privilégier pour un suivi temporel.

```

1 lake_results = earthaccess.search_data(
2     short_name='SWOT_L2_HR_LakeSP_D',
3     granule_name='*Prior*_214_NA*') # 'Obs', 'Prior' ou 'Unassigned'
4
5 earthaccess.download([lake_results[-1]], "./data_downloads")
6
7 filename2 = earthaccess.results.DataGranule.data_links(lake_results[-1], access='external')
8 filename2 = filename2[0].split("/)[-1]
9
10 with zipfile.ZipFile(f'data_downloads/{filename2}', 'r') as zip_ref:
11     zip_ref.extractall('data_downloads')
12
13 SWOT_HR_shp2 = gpd.read_file(f"data_downloads/{filename2.replace('.zip', '.shp')}")
14 SWOT_HR_shp2

```

1.7.1 Élévation des lacs

Les polygones étant peu nombreux sur une petite emprise, on peut annoter chaque lac avec sa valeur de WSE en plaçant l'étiquette sur le centroïde.

```

1 # Lac Oromocto (N.-B.)
2 SWOT_HR_shp2_clipped = gpd.clip(
3     SWOT_HR_shp2.query('wse != -99999999999'), box(-67.10, 45.47, -66.82, 45.65))
4

```

```

5 fig, ax = plt.subplots(figsize=(7, 7))
6 SWOT_HR_shp2_clipped.plot(
7     ax=ax, column='wse', cmap='cool', legend=True,
8     legend_kwds={'label': "Élévation de la surface d'eau (m)", 'orientation': "vertical"})
9
10 centroids = SWOT_HR_shp2_clipped.geometry.centroid
11 for x, y, label in zip(centroids.x, centroids.y, SWOT_HR_shp2_clipped['wse']):
12     ax.text(x, y, f'{label:.2f}', fontsize=8, ha='center', va='center', color='black')
13
14 cx.add_basemap(ax, crs=SWOT_HR_shp2_clipped.crs, source=cx.providers.Esri.WorldStreetMap)
15 plt.show()

```

1.7.2 Fraction d'eau sombre (*dark water*)

L'attribut `dark_frac` donne la fraction de la superficie totale du lac (`area_total`) classée comme **eau sombre**, entre 0 (aucune) et 1 (100 %). L'eau sombre correspond à une surface trop lisse, qui réfléchit le signal radar loin de l'antenne : peu d'énergie revient et la surface risque de ne pas être détectée comme de l'eau. C'est un indicateur clé de fiabilité.

```

1 fig, ax = plt.subplots(figsize=(7, 7))
2 SWOT_HR_shp2_clipped.plot(
3     ax=ax, column='dark_frac', cmap='RdYlGn_r', legend=True,
4     legend_kwds={'label': "Fraction d'eau sombre", 'orientation': "vertical"})
5
6 centroids = SWOT_HR_shp2_clipped.geometry.centroid
7 for x, y, label in zip(centroids.x, centroids.y, SWOT_HR_shp2_clipped['dark_frac']):
8     ax.text(x, y, f'{label:.2f}', fontsize=8, ha='center', va='center', color='black')
9
10 cx.add_basemap(ax, crs=SWOT_HR_shp2_clipped.crs, source=cx.providers.Esri.WorldStreetMap)
11 plt.show()

```

1.8 Produit nuage de points L2_HR_PIXC

Les produits qui suivent sont stockés en **NetCDF** et non en **shapefile** : on les ouvre avec **xarray** plutôt qu'avec **geopandas**, et aucune décompression n'est nécessaire.

```

1 pixc_results = earthaccess.search_data(
2     short_name='SWOT_L2_HR_PIXC_D', # 'SWOT_L2_HR_PIXC_2.0' pour la version C
3     granule_name='*_214_074L*') # passe, tuile et côté de fauchée (L ou R)
4     # bounding_box=(-72.73, 46.58, -72.60, 46.62) # filtrage par emprise
5
6 earthaccess.download([pixc_results[-1]], "./data_downloads")

```

Le fichier comporte trois groupes (`pixel_cloud`, `tv`, `noise`); il faut donc préciser le **groupe** à l'ouverture, sans quoi `xarray` ne voit ni les coordonnées ni les variables.

```
1 ds_PIXC = xr.open_mfdataset(
2     "data_downloads/SWOT_L2_HR_PIXC_*.nc", group='pixel_cloud', engine='h5netcdf')
3 ds_PIXC
```

Le nuage de points n'étant pas sur une grille, il s'affiche avec un nuage de dispersion. On borne l'échelle de couleur sur les 2^e et 98^e percentiles pour éviter que quelques valeurs extrêmes n'écrasent le contraste.

```
1 vmin, vmax = np.nanpercentile(ds_PIXC.height, [2, 98])
2
3 fig, ax = plt.subplots(figsize=(10, 10))
4 cax = ax.scatter(ds_PIXC.longitude, ds_PIXC.latitude, c=ds_PIXC.height,
5                 s=1, vmin=vmin, vmax=vmax, cmap='viridis', rasterized=True)
6 cbar = fig.colorbar(cax, ax=ax, shrink=0.5)
7 cbar.set_label('Hauteur ellipsoïdale (m)')
8 cx.add_basemap(ax, crs='EPSG:4326', source=cx.providers.Esri.WorldStreetMap)
9 plt.show()
```

💡 Astuce

Rastériser les nuages de points denses

Un produit `PIXC` contient des millions de points. Sans `rasterized=True`, chaque point devient un objet vectoriel dans le PDF exporté, qui peut alors atteindre plusieurs dizaines de mégaoctets. L'option convertit le tracé en image tout en gardant les axes et les étiquettes en vecteur.

1.8.1 Découpage et export vers QGIS

QGIS ne sait pas ouvrir directement un `PIXC`. On filtre donc les points sur une emprise **et** sur la classification (les classes 3 et 4 correspondent à l'eau), puis on exporte en `shapefile`.

```
1 logging.getLogger('fiona').setLevel(logging.ERROR)
2
3 lat_min, lat_max = 45.88, 45.98
4 lon_min, lon_max = -66.77, -66.49
5
6 lat = np.asarray(ds_PIXC.latitude[:])
7 lon = np.asarray(ds_PIXC.longitude[:])
8 classif = np.asarray(ds_PIXC.classification[:])
9
10 # Masque combiné : emprise géographique et pixels classés « eau »
11 mask = ((lat > lat_min) & (lat < lat_max) &
12         (lon > lon_min) & (lon < lon_max) &
13         (classif > 2) & (classif < 5))
```

```

14
15 df = pd.DataFrame({
16     'height': np.asarray(ds_PIXC.height[:])[mask],
17     'classif': classif[mask],
18     'latitude': lat[mask],
19     'longitude': lon[mask],
20 })
21
22 points = [Point(x, y) for x, y in zip(df.longitude, df.latitude)]
23 gdf_out = gpd.GeoDataFrame(df, geometry=points, crs="EPSG:4326")
24 gdf_out.to_file('./data_downloads/PIXC_clipped.shp')

```

1.9 Produit L2_HR_PIXCVec

Le **PIXCVec** accompagne le **PIXC** : mêmes pixels, mais avec une géolocalisation améliorée. Ses variables portent le suffixe **_vectorproc**.

```

1 pixcvec_results = earthaccess.search_data(
2     short_name='SWOT_L2_HR_PIXCVEC_D',
3     granule_name='*_214_074L*')
4
5 earthaccess.download([pixcvec_results[-1]], "./data_downloads")
6
7 ds_PIXCVEC = xr.open_mfdataset(
8     "data_downloads/SWOT_L2_HR_PIXCVec*.nc", decode_cf=False, engine='h5netcdf')
9 ds_PIXCVEC

```

Avec **decode_cf=False**, les **valeurs de remplissage ne sont pas converties** en **NaN** automatiquement : il faut les masquer à la main avant de tracer.

```

1 pixcvec_htvals = ds_PIXCVEC.height_vectorproc.compute()
2 pixcvec_latvals = ds_PIXCVEC.latitude_vectorproc.compute()
3 pixcvec_lonvals = ds_PIXCVEC.longitude_vectorproc.compute()
4
5 # Remplacer les valeurs de remplissage par des NaN
6 pixcvec_htvals[pixcvec_htvals > 15000] = np.nan
7 pixcvec_latvals[pixcvec_latvals < 1] = np.nan
8 pixcvec_lonvals[pixcvec_lonvals > -1] = np.nan
9
10 vmin, vmax = np.nanpercentile(pixcvec_htvals, [2, 98])
11
12 fig, ax = plt.subplots(figsize=(10, 10))
13 cax = ax.scatter(pixcvec_lonvals, pixcvec_latvals, c=pixcvec_htvals,
14                 s=1, vmin=vmin, vmax=vmax, cmap='viridis', rasterized=True)

```

```

15 cbar = fig.colorbar(cax, ax=ax, shrink=0.5)
16 cbar.set_label('Hauteur ellipsoïdale (m)')
17 cx.add_basemap(ax, crs='EPSG:4326', source=cx.providers.Esri.WorldStreetMap)
18 plt.show()

```

1.10 Produit matriciel L2_HR_Raster

Le produit **Raster** est découpé en scènes et décliné en deux résolutions. Le motif ***100m*_214_037F*** sélectionne la résolution de 100 m, la passe 214 et la scène 037.

```

1 raster_results = earthaccess.search_data(
2     short_name='SWOT_L2_HR_Raster_D',
3     granule_name='*100m*_214_037F*') # résolution (100m ou 250m), passe, scène
4
5 earthaccess.download([raster_results[-1]], "./data_downloads")
6
7 ds_raster = xr.open_mfdataset('data_downloads/SWOT_L2_HR_Raster*', engine='h5netcdf')
8 ds_raster

```

Le produit étant déjà sur une grille régulière, **hvplot** permet un affichage interactif (zoom, survol) en une ligne.

```

1 hv.extension('bokeh', 'matplotlib')
2 hv.output(ds_raster['wse'].hvplot.image(y='y', x='x'))

```

1.10.1 Masquage par l'indicateur de qualité

L'indicateur **wse_qual** suit la même échelle que **node_q** (0 bon, 1 suspect, 2 dégradé, 3 mauvais). La méthode **where** de **xarray** remplace par **NaN** les pixels qui ne satisfont pas la condition, sans modifier la grille.

```

1 masked_variable = ds_raster['wse'].where(ds_raster['wse_qual'] < 3)
2 masked_variable

```

1.10.2 Découpage sur une région d'intérêt

rioxarray ajoute à **xarray** les opérations géospatiales. Il faut d'abord **déclarer** les dimensions spatiales et le système de coordonnées, puis reprojeter l'emprise dans ce même système avant de découper.

```

1 ROI = box(-67.28, 45.72, -66.22, 46.20)
2 bbox_gdf = gpd.GeoDataFrame({'geometry': [ROI]}, crs='EPSG:4326')
3
4 masked_variable.rio.set_spatial_dims(x_dim="x", y_dim="y", inplace=True)
5 masked_variable.rio.write_crs("epsg:32619", inplace=True)
6
7 # L'emprise doit être dans le même système que la grille avant le découpage
8 bbox_gdf = bbox_gdf.to_crs("epsg:32619")
9 clipped = masked_variable.rio.clip(bbox_gdf.geometry.apply(mapping), drop=True)
10
11 # L'attribut 'grid_mapping' empêche l'écriture NetCDF : on le retire
12 clipped.attrs.pop('grid_mapping', None)
13
14 os.makedirs('./clip_data', exist_ok=True)
15 clipped.to_netcdf('./clip_data/clipped_raster.nc')

```

⚠ Attention

Vérifier la zone UTM

Le code EPSG de la grille UTM dépend de la scène : **EPSG:32619** correspond à la zone 19 Nord, **EPSG:32618** à la zone 18 Nord. Lisez la zone dans les attributs du fichier (`ds_raster.attrs`) plutôt que de la coder en dur, sinon le découpage produira un résultat vide ou décalé de centaines de kilomètres.

1.11 Séries temporelles avec Hydrocron

Hydrocron est une API du PO.DAAC qui agrège les observations SWOT successives d'un même tronçon, nœud ou lac, et renvoie une série temporelle en CSV ou GeoJSON. C'est de loin le moyen le plus simple d'obtenir un historique sans télécharger un granule par date.

Les identifiants (`reach_id`, `node_id`, `lake_id`) se lisent dans les produits **RiverSP/LakeSP** de la section 1.6, ou se cherchent dans [SWORD Explorer](#).

⚠ Attention

Identifiants et versions

Les identifiants de nœuds et de tronçons **diffèrent entre les versions C et D**. Un identifiant issu d'un produit version C interrogé sur la collection version D renverra une série vide ou, pire, la série d'un autre objet.

1.11.1 Localiser un tronçon de rivière

```

1 feature = "Reach"
2 feature_id = "72608300043"

```

```

3 start_time = "2023-08-01T00:00:00Z"
4 end_time = "2026-01-21T00:00:00Z"
5 collection_name = "SWOT_L2_HR_RiverSP_D"
6
7 parameters = (
8     "https://soto.podaac.earthdatacloud.nasa.gov/hydrocron/v1/timeseries?"
9     + "feature=" + feature
10    + "&feature_id=" + feature_id
11    + "&start_time=" + start_time
12    + "&end_time=" + end_time
13    + "&collection_name=" + collection_name
14    + "&output=geojson"
15    + "&fields=reach_id,time_str,wse,width,cycle_id"
16 )
17
18 hydrocron_response = requests.get(parameters).json()
19 geojson_data = hydrocron_response['results']['geojson']
20
21 carte = folium.Map(tiles="cartodbpositron", width=700, height=700)
22 folium.GeoJson(geojson_data, name='Tronçon SWOT').add_to(carte)
23 folium.LayerControl().add_to(carte)
24 carte.fit_bounds(carte.get_bounds(), padding=(5, 5))
25 carte

```

1.11.2 Série temporelle de niveau d'eau à un nœud

La sortie `csv` se charge directement dans un `DataFrame`. Deux filtres sont indispensables : écarter les enregistrements `no_data` et retenir les nœuds de qualité acceptable (`node_q < 3`).

```

1 feature = "Node"
2 feature_id = "72608300050213" # rivière Saint-Jean, près de Fredericton
3 collection_name = "SWOT_L2_HR_RiverSP_D"
4
5 parameters = (
6     "https://soto.podaac.earthdatacloud.nasa.gov/hydrocron/v1/timeseries?"
7     + "feature=" + feature
8     + "&feature_id=" + feature_id
9     + "&start_time=2023-08-01T00:00:00Z"
10    + "&end_time=2026-01-21T00:00:00Z"
11    + "&collection_name=" + collection_name
12    + "&output=csv"
13    + "&fields=reach_id,node_id,time_str,node_q,wse,width,cycle_id"
14 )
15
16 hydrocron_response = requests.get(parameters).json()

```

```

17 df = pd.read_csv(StringIO(hydrocron_response['results']['csv']))
18
19 df = df[df['time_str'] != 'no_data']
20 df['time_str'] = pd.to_datetime(df['time_str'], format='%Y-%m-%dT%H:%M:%SZ')
21 df = df[df['node_q'] < 3]
22
23 fig = plt.figure(figsize=(15, 5))
24 plt.plot(df['time_str'], df['wse'], marker='o', linestyle='None')
25 plt.ylabel("Élévation de la surface d'eau (m)")
26 plt.xlabel("Date d'observation SWOT")
27 plt.title(f"WSE Hydrocron pour le nœud {df['node_id'].iloc[0]}")
28 plt.show()

```

1.11.3 Série temporelle pour un lac

Pour les lacs, on filtre en plus sur `quality_f` (0 = bon) et sur `dark_frac` (moins de 50 % d'eau sombre).

```

1 parameters = (
2     "https://soto.podaac.earthdatacloud.nasa.gov/hydrocron/v1/timeseries?"
3     + "feature=PriorLake"
4     + "&feature_id=7260055912"          # lac Oromocto
5     + "&start_time=2023-08-31T00:00:00Z"
6     + "&end_time=2026-01-21T00:00:00Z"
7     + "&collection_name=SWOT_L2_HR_LakeSP_D"
8     + "&output=csv"
9     + "&fields=lake_id,time_str,wse,area_total,quality_f,dark_frac"
10 )
11
12 hydrocron_response = requests.get(parameters).json()
13 df = pd.read_csv(StringIO(hydrocron_response['results']['csv']))
14
15 df = df[df['time_str'] != 'no_data']
16 df = df[df['quality_f'] == 0]          # ne garder que les observations de bonne qualité
17 df = df[df['dark_frac'] < 0.5]        # écarter les scènes majoritairement en eau sombre
18 df['time_str'] = pd.to_datetime(df['time_str'], format='%Y-%m-%dT%H:%M:%SZ')
19
20 fig = plt.figure(figsize=(15, 5))
21 plt.plot(df['time_str'], df['wse'], marker='o', linestyle='None')
22 plt.ylabel("Élévation de la surface d'eau (m)")
23 plt.xlabel("Date d'observation SWOT")
24 plt.title(f"WSE Hydrocron pour le lac {df['lake_id'].iloc[0]}")
25 plt.show()

```


Note**Position des lacs dans Hydrocron**

En raison de la taille des polygones du produit L2_HR_LakeSP, Hydrocron ne renvoie que le **point central** du lac, pour respecter les spécifications GeoJSON. Ces centroides ne doivent pas être considérés comme des positions exactes.

1.12 Profil en long de la surface d'eau

Un **profil en long** trace l'élévation de la surface d'eau en fonction de la distance parcourue le long du cours d'eau. Il se construit en ordonnant les nœuds d'un tronçon par **node_id** et en cumulant l'attribut **p_length** (la longueur associée à chaque nœud dans SWORD).

```

1 def download_data(pass_numbers, continent_code, path, temporal_range):
2     """Recherche et téléchargement des fichiers Node RiverSP pour une liste de passes."""
3     links_list = []
4     for pass_num in pass_numbers:
5         river_results = earthaccess.search_data(
6             short_name='SWOT_L2_HR_RIVERSP_D',
7             temporal=temporal_range,
8             granule_name=f"*Node*_{pass_num}_{continent_code}*")
9         links_list.extend(
10             earthaccess.results.DataGranule.data_links(r, access='external')[0]
11             for r in river_results)
12     earthaccess.download(links_list, path)
13     return links_list
14
15
16 def extract_files(links_list, path):
17     """Décompresse les archives téléchargées et renvoie leurs noms."""
18     filenames = [link.split("/")[-1] for link in links_list]
19     for filename in filenames:
20         with zipfile.ZipFile(f"{path}/{filename}", 'r') as zip_ref:
21             zip_ref.extractall(path)
22     return filenames
23
24
25 def load_shapefiles(filenames, path):
26     """Concatène tous les shapefiles en un seul GeoDataFrame."""
27     shps = [filename.replace('zip', 'shp') for filename in filenames]
28     return gpd.GeoDataFrame(
29         pd.concat([gpd.read_file(f"{path}/{shp}") for shp in shps], ignore_index=True))

```

Le filtrage retient les nœuds compris entre le nœud aval et le nœud amont, écarte les valeurs de remplissage et, au besoin, restreint à une date d'acquisition.

```

1 def filter_data_by_nodes(SWOT_HR_df, up_node, dn_node, date=None):
2     filtered = SWOT_HR_df[
3         (SWOT_HR_df['node_id'] >= dn_node) &
4         (SWOT_HR_df['node_id'] < up_node) &
5         (SWOT_HR_df['wse'] != -9999999999)]
6     if date:
7         filtered = filtered[filtered['time_str'].str.contains(date)]
8     return filtered.sort_values(['node_id'])
9
10
11 def calculate_distances(profil):
12     """Distance cumulée le long du profil, à partir de la longueur de chaque nœud."""
13     return np.concatenate([[0.0], np.cumsum(profil['p_length'].values[:-1])])
14
15
16 def plot_profile(profil, dist, label):
17     fig, ax = plt.subplots(figsize=(15, 5))
18     ax.plot(dist, profil['wse'], marker='o', linestyle='None', label=label)
19     ax.set_xlabel('Distance cumulée (m)')
20     ax.set_ylabel("Élévation de la surface d'eau (m)")
21     ax.legend()
22     plt.show()

```

Il ne reste qu'à renseigner la zone et la période d'intérêt, puis à enchaîner les étapes.

```

1 path = './profil_node'
2 pass_number = ["214"]
3 continent_code = "NA" # NA pour l'Amérique du Nord
4 temporal_range = ('2025-05-24 00:00:00', '2025-05-27 23:59:59')
5 dn_node = 72608300270021 # nœud aval (rivière Nashwaak)
6 up_node = 72608300280071 # nœud amont
7
8 os.makedirs(path, exist_ok=True)
9 links_list = download_data(pass_number, continent_code, path, temporal_range)
10 filenames = extract_files(links_list, path)
11
12 SWOT_HR_df = load_shapefiles(filenames, path)
13 SWOT_HR_df['node_id'] = SWOT_HR_df['node_id'].astype(float)
14
15 profile_1 = filter_data_by_nodes(SWOT_HR_df, up_node, dn_node, date='2025-05-26')
16 dist_1 = calculate_distances(profile_1)
17 plot_profile(profile_1, dist_1, '26 mai 2025')

```

1.13 Changement de référentiel vertical avec csrspsy

Le référentiel de SWOT n'est pas celui du Canada : comparer une WSE SWOT à un relevé de station exige la chaîne de conversion décrite plus haut. La bibliothèque **csrspsy** encapsule les grilles de transformation de Ressources naturelles Canada. L'exemple ci-dessous convertit vers le référentiel vertical du Nouveau-Brunswick (NAD83(SCRS), époque 2010, géoïde CGG2013).

```
1 from csrspsy.main import CSRSTransformer
2 from csrspsy.enums import CoordType, Reference, VerticalDatum
3 from csrspsy.utils import sync_missing_grid_files
```

On commence par découper la zone d'intérêt : chaque province pouvant adopter une époque et un référentiel différents, la transformation doit être appliquée province par province.

```
1 shapefile_path =
  ↳ 'data_downloads/SWOT_L2_HR_RiverSP_Node_044_214_NA_20260111T003728_20260111T004317_PID0_01.zip'
2 points_gdf = gpd.read_file(shapefile_path)
3
4 bounding_box = box(-67.28, 45.72, -66.22, 46.20)
5 gdf_nodes = points_gdf[points_gdf.geometry.within(bounding_box)].copy()
```

L'époque source est la **date d'acquisition** du granule, exprimée en année décimale.

```
1 def decimal_year(dt):
2     """Convertit une date en année décimale (ex. 2026-01-11 -> 2026.028)."""
3     year_start = datetime(dt.year, 1, 1)
4     year_end = datetime(dt.year + 1, 1, 1)
5     return dt.year + (dt - year_start).total_seconds() / (year_end - year_start).total_seconds()
6
7 acquisition_date = "2026-01-11" # date d'acquisition du granule à convertir
8 s_epoch = decimal_year(datetime.strptime(acquisition_date, "%Y-%m-%d"))
```

On repasse ensuite de la WSE à la hauteur ellipsoïdale ($h = wse + geoid_hght$), avant de transformer chaque point.

```
1 sync_missing_grid_files() # télécharge les grilles de transformation manquantes
2
3 gdf_nodes['h'] = gdf_nodes['wse'] + gdf_nodes['geoid_hght']
4
5 transformer = CSRSTransformer(
6     s_ref_frame=Reference.ITRF14, # référentiel source : celui de SWOT
7     t_ref_frame=Reference.NAD83SCRS, # référentiel cible : canadien
8     s_coords=CoordType.GEOG,
9     t_coords=CoordType.GEOG,
```

```

10     s_epoch=s_epoch,                # époque source : date d'acquisition
11     t_epoch=2010,                  # époque cible : Nouveau-Brunswick
12     epoch_shift_grid='ca_nrc_NAD83v70VG.tif',
13     s_vd=VerticalDatum.WGS84,
14     t_vd=VerticalDatum.CGG2013)
15
16 transformed = [
17     list(transformer([(row.lon, row.lat, row.h)]))[0][2]
18     for row in tqdm(gdf_nodes.itertuples(), total=len(gdf_nodes), desc="Transformation")]
19
20 gdf_nodes['wse_transformed'] = transformed
21
22 os.makedirs('./ref_change', exist_ok=True)
23 gdf_nodes.to_file('./ref_change/nodes_transformed.shp')

```

1.14 Défis et limites

Le budget d'erreur de SWOT combine des effets de propagation (ionosphère, troposphère), des effets liés à la surface observée et des effets liés à la géométrie de visée.

Le chatouillement spéculaire (*specular ringing*). Une surface d'eau très lisse se comporte comme un miroir : le retour est extrêmement fort à un endroit et provoque des artefacts en anneaux dans l'image radar.

L'eau sombre (*dark water*). Le phénomène inverse : en l'absence de vent et de vagues, la surface dévie le signal loin de l'antenne et **très peu d'énergie revient** au radar. La surface est alors mal classée, voire non détectée. L'eau sombre est plus fréquente aux grandes distances au nadir (**xtrk_dist** supérieur à 40 km) et, en deçà, lorsque les vitesses de vent sont faibles.

Le repliement (*layover*). Un relief ou un ouvrage proche du plan d'eau peut renvoyer un écho à la même distance oblique que la surface d'eau, contaminant la mesure.

La géolocalisation. L'exactitude de la position dépend de la connaissance de l'attitude et de l'orbite du satellite; les points de croisement (*cross-over*) servent à corriger ces dérives.

Les bases a priori. SWORD et le PLD ne sont pas parfaits : polygones de lacs mal délimités (le lac Témiscamingue, par exemple, a dû être corrigé manuellement), tronçons traversant des îles, décalages entre les nœuds SWORD et le lit réel. Comme les produits **RiverSP** et **LakeSP** **agrègent les pixels selon ces bases**, une erreur dans la base se propage directement dans la mesure. Les nœuds associés à des lacs ou à des barrages (**node_id** se terminant par 3 ou 4) présentent une proportion nettement plus élevée de **node_q = 3**.

La glace. La glace de rivière et de lac modifie profondément la réponse radar. C'est à la fois une limite (les mesures hivernales sont à interpréter avec prudence, filtrer avec **ice_clim_f**) et une **occasion** : SWOT permet de suivre l'évolution du couvert de glace à une fréquence inaccessible aux capteurs optiques, souvent bloqués par les nuages.

 Aller plus loin**Exactitude de la WSE aux nœuds au Canada**

Une évaluation menée à l'Université de Sherbrooke sur 110 stations limnimétriques nivelées au Canada donne, pour l'ensemble des données, un écart absolu de WSE au 68^e percentile de 0,42 m en version C et de 0,25 m en version D (orbite *Science*). En ne retenant que les observations sans glace (`ice_clim_f = 0`) et de qualité acceptable (`node_q < 3`), l'écart descend à 0,23 m (version C) et 0,21 m (version D).

Autrement dit : la version D apporte un gain de quelques centimètres, et **le filtrage par les indicateurs de qualité apporte davantage que le changement de version**. C'est le message pratique le plus important de ce chapitre.

 Exercice**Exercices de révision**

Exercice 1. Un collègue vous transmet un extrait de **RiverSP** dans lequel la colonne `wse` contient des valeurs autour de -10^{12} . Que représentent ces valeurs et comment les traiter?

Correction

Il s'agit de la valeur de remplissage `-999999999999`, utilisée lorsque le nœud n'a pas pu être mesuré. Elle doit être écartée par filtrage (`df[df['wse'] != -999999999999]`) et **non** remplacée par zéro ni incluse dans une moyenne.

Exercice 2. Vous disposez de la WSE d'un nœud SWOT et vous souhaitez la comparer au relevé d'une station hydrométrique du Nouveau-Brunswick. Énumérez les trois différences de référentiel à réconcilier.

Correction

Le **datum** (ITRF2014 pour SWOT contre NAD83(SCRS) pour la station), l'**époque** (date d'acquisition contre 2010) et le **géοide** (EGM2008 *mean-tide* contre CGVD2013 *tide-free*).

Exercice 3. Vous cherchez à suivre l'évolution du niveau d'un lac sur deux ans. Quel produit et quel type de fichier choisissez-vous, et pourquoi?

Correction

Le produit **L2_HR_LakeSP**, fichier **Prior** : les identifiants `lake_id` y sont issus de la base PLD et restent stables d'une date à l'autre, ce qui permet de construire une série temporelle. Les fichiers **Obs** et **Unassigned** utilisent des identifiants d'observation qui changent à chaque passage. En pratique, l'API Hydrocron avec `feature=PriorLake` évite d'avoir à télécharger un granule par date.

Exercice 4. Un lac apparaît avec `dark_frac = 0.85`. Faut-il retenir cette observation?

Correction

Non. 85 % de la superficie du lac est classée comme eau sombre : la surface était probablement trop lisse pour renvoyer un signal exploitable, et l'étendue comme l'élévation mesurées sont peu fiables. Un seuil usuel est d'écarter les observations dont `dark_frac` dépasse 0,5.

1.15 Ressources

- Notes techniques de tous les produits SWOT : <https://podaac.jpl.nasa.gov/SWOT?tab=datasets-information§ions=about>
- Earthdata Search : https://search.earthdata.nasa.gov/search?q=SWOT_L2_HR
- Le site de la mission (NASA) : <https://swot.jpl.nasa.gov/>
- Visualiseur de fauchée, pour trouver la passe et la date de survol : <https://swot.jpl.nasa.gov/mission/swath->

[visualizer/](#)

- Tutoriels du PO.DAAC Cookbook : https://podaac.github.io/tutorials/quarto_text/SWOT.html
- Dépôt SWOT-Canada (couches de passes/tuiles, ateliers) : <https://github.com/sfoucher/SWOT-Canada>
- SWORD Explorer, pour trouver un **reach_id** ou un **node_id** : <https://www.swordexplorer.com/>

Ce chapitre s'appuie sur l'atelier *SWOT and Height Reference* présenté par Mélanie Trudel, Gabriela Siles, Samuel Foucher et Victoria Litalien à la conférence mi-mandat de l'ACRH (CWRA) du 29 janvier 2026, ainsi que sur le tutoriel *SWOT Hydrology Dataset Exploration on a Local Machine* du PO.DAAC (Cassie Nickles et collaborateurs), adapté par les équipes de l'Université de Sherbrooke et de l'Université Laval.

Bibliographie